

Predicate

- = interfață funcțională gata făcută în Java.
- = testează ceva și dă rezultatul true/false.

* metoda abstractă: boolean test(T t)

* în loc să definim mai @FunctionalInterface..., putem să scriem direct funcția lambda cu Predicate

exemplu ①:

```
Predicate <Student> promovat = x -> x.getMedia() >= 5;
```

↳ definim predicatul

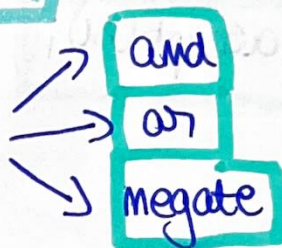
```
Student s = new Student(...
```

```
boolean rez = promovat.test(s);
```

se apelează
(nuu avea
true/false).

exemplu ②:

putem folosi operații logice



// primul predicat

```
Predicate <Student> promovat = x -> x.getMedia() >= 5;
```

// al doilea

```
Predicate <Student> inceput = x -> x.getNum().startsWith("A");
```

// le combinăm student-bun

```
Predicate <Student> = promovat.and(inceput);
```

// negare

```
Predicate <Student> = negate(promovat);
```


Consumer (consumator)

→ Predicate Îmbreabă, iar Consumer execută.

→ Primesc un obiect și face ceva cu el, dar nu returnează nimic.

* metoda abstractă: `void accept(T t)`

exemplu ①

```
public class Main {  
    public static void main (String[] args) {
```

```
        Consumer < Student > afisator = s -> System.out.println  
        ("Student " + ...);
```

```
        Student s1 = new Student (...);
```

```
        afisator.accept(s1);  
    }  
}
```

exemplu ② → putem face 2 acțiuni, una după alta
folosind andThen

```
Consumer < Student > afisator = s -> System.out.println ... ;  
Consumer < Student > felicitare = s -> System.out.println  
    ("a primit bursa");  
    2 consumatori
```

```
Consumer < Student > complet = afisator.andThen(felicitare);
```

```
complet.accept(s1);
```

student definit sus

Metode adiționale colecțiilor (Collection)

List * forEach

→ metoda `forEach` a listelor așteaptă ca parametru un `Consumer`.

→ o listă de studenți

`List < Student > lista = Arrays.asList(new Student("Ana" ...`

// Mem să îi afișăm pe toți

// varianta 1: lambda explicit

// `x → System.out.println(x)` este un `Consumer` (ia x

// și nu returnează nimic

`studenti.forEach(x → System.out.println(x));`

// varianta 2: referință la metodă

`studenti.forEach(System.out::println);`

// folosim un predicat și o funcție de if

`Predicate < Student > estePromovat = x → x.getMedie() >= 5;`

`list.forEach(x → { if (estePromovat.test(x))
System.out.println(x); });`

↳ îl afișăm doar dacă este promovat.

* removeIf

→ ștergem din listă toți cei care nu sunt promovați

`list.removeIf(estePromovat.negate());`

Exerciții

- Să se șteargă dintr-o listă de șiruri de caractere, toate elementele care încep cu “a”.
- Să se șteargă dintr-o listă de șiruri de caractere, toate elementele care sunt prefixele unui cuvânt. De exemplu “Anamaria”. Folositi funcție lambda si referință la metodă.
- Să se șteargă dintr-o listă de șiruri de caractere, toate elementele care conțin șirul vid. Folositi funcție lambda si referință la metodă.
- Să se șteargă toți studenții a căror nume începe cu “B”, dintr-o listă de studenți.

Exerciții din curs (Cursul, Slide 21)

1. Să se ștergă dintr-o listă de șiruri de caractere, toate elementele care încep cu "a" sau "A". Apoi să se afișeze pe ecran.

* metoda 1 → cu mai multe predicate separate

```
Predicate <String> incepecuA = x → x.startsWith("A");
```

```
Predicate <String> incepecuA = x → x.startsWith("a");
```

```
Predicate <String> filtru = incepecuA.or(incepecuA);
```

List <String> lista → o listă cu string-uri

```
lista.removeIf(filtru);
```

```
lista.forEach(System.out::println);
```

* metoda 2 → cu un singur removeIf și funcție lambda definită direct

```
lista.removeIf(x → x.startsWith("a") || x.startsWith("A"));
```

② Să se ștergă dintr-o listă de șiruri toate elem. care sunt prefixele unui cuvânt. Folosiți funcție lambda și referință la metodă.

varianta marcată (fără referință la metodă)

```
lista.removeIf(x → "Anamaria".startsWith(x));
```

Subiectul

metoda

⇒ răspunsul final:

```
lista.removeIf("Anamaria"::startsWith);
```


3) Să se ștergă toate elem. care conțin șirul vid.
(funcție lambda + referința metodă).

varianta simplă

```
lista.removeIf(x -> x.contains(""));
```

varianta cu referință la metodă (din altă clasă) este în clasă Verificari

→ trebuie să ne facem o metodă nouă

```
public static boolean are_vid(String s) {  
    if (s.contains("")) return true;  
    else return false;  
}
```

```
lista.removeIf(Verificari::are_vid);
```

varianta cu referință la metodă care folosește un predicat

(ștergem dacă are litera x)

```
public static boolean are_x(String s) {
```

```
    Predicate<String> arex = x -> x.contains(x) || x.contains(x);
```

```
    return arex.test(s);
```

```
}
```

```
lista.removeIf(Verificari::are_x);
```

x mare
sau x
mic

4) Toti studentii a caror nume incepe cu B

(presupunem ca avem o clasa Student)

```
lista.removeIf(x -> x.getName().startsWith("B"));
```

```
lista.forEach(System.out::println);
```

↳ va afisa toti studentii

```
if (2 print?) list.removeIf(x -> x.getName().startsWith("B"));
```

```
list.removeIf(x -> x.getName().startsWith("B"));
```

```
list.removeIf(x -> x.getName().startsWith("B"));
```

```
list.removeIf(x -> x.getName().startsWith("B"));
```